

LEAGUE OF ARROW

Taller de base de datos

Integrantes:

Jordán Arias

Ronald Garrido

Fernando Lazo

Martín Oviedo

Johanny Simken

Sofía Vergara

Profesor: Matías Calderón
Ayudante: Fernando Solís

T A B L A D E C O N T E N I D O S

Contenidos del Proyecto

- 01. Contexto y Objetivos
- 02. Arquitectura del Sistema
- 03. Modelo de Base de Datos
- 04. Triggers
- 05. Stored Procedures
- 06. Vista Materializada
- 07. Api Rest y Seguridad
- 08. Ejecucion de la aplicación
- 09. Conclusiones

CONTEXTO Y OBJETIVOS

Problemática

Se necesita crear una plataforma para gestionar competencias de

tiro con arco, donde se deberá visualizar:

- Arqueros y sus categorías
- Torneos activos con suscripciones
- Registro de rondas y puntuaciones por fecha
- Tabla de posiciones y leaderboard historico

Objetivos

Los objetivos que se deberan de cumplir son los siguientes:

- Diseñar esquemas relacionales normalizados
- Implementar Triggers y Stored Procedures
- Desarrollar API REST segura (JWT + RBAC)
- Optimizar consultas con indices y vistas materializadas

AVANCE DEL PROYECTO

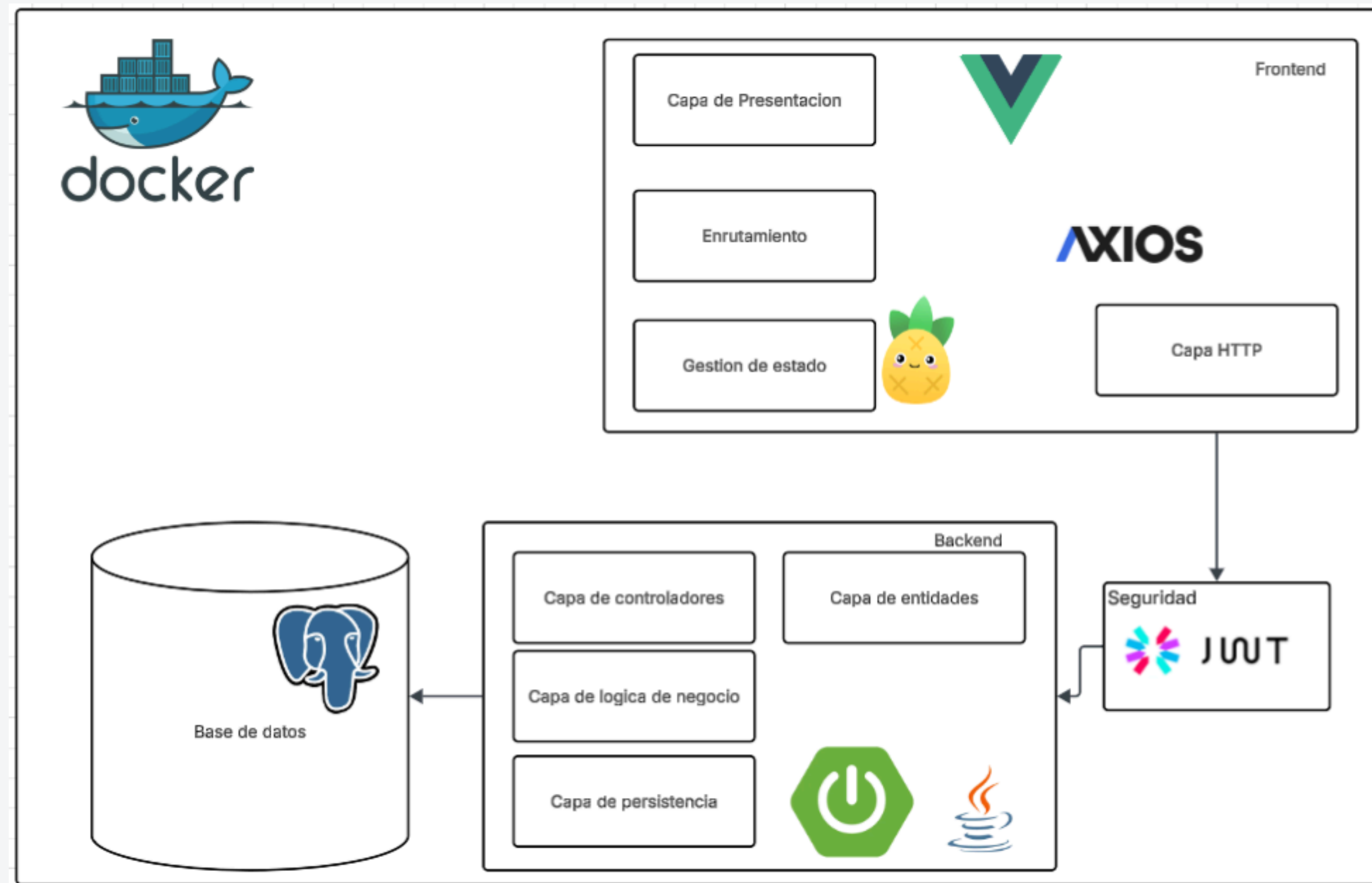
-Base de Datos: Desarrollo de estructura de la base de datos, triggers, indexaciones, procedimientos almacenados, vistas materializadas

-Backend: Desarrollo de arquitectura en capas, consultas SQL, implementan de JWT, desarrollo de CRUD's y API's

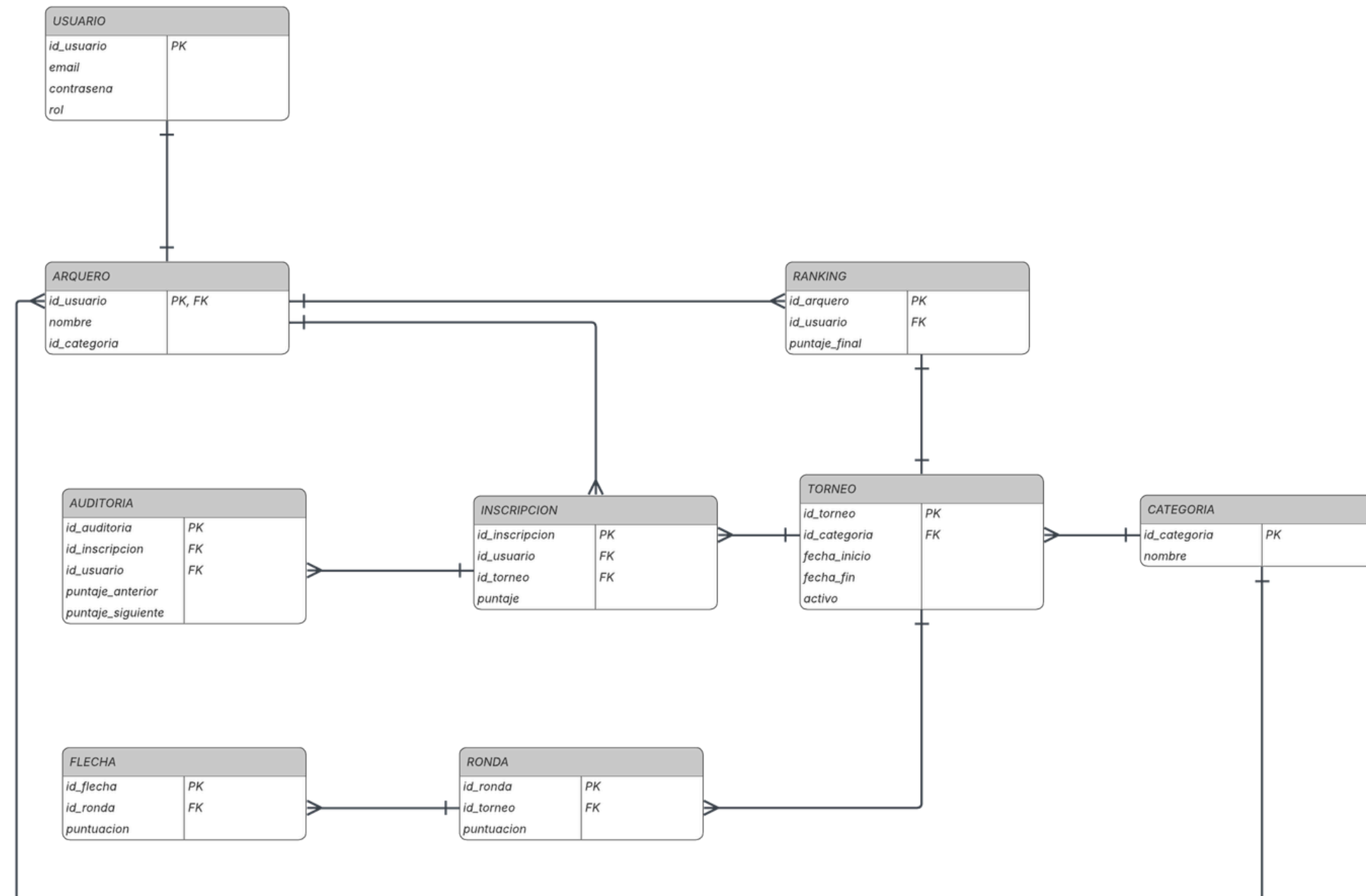
-Frontend: Implementación funcional casi completa, con un trigger por integrar.

El proyecto esta listo en un 95%

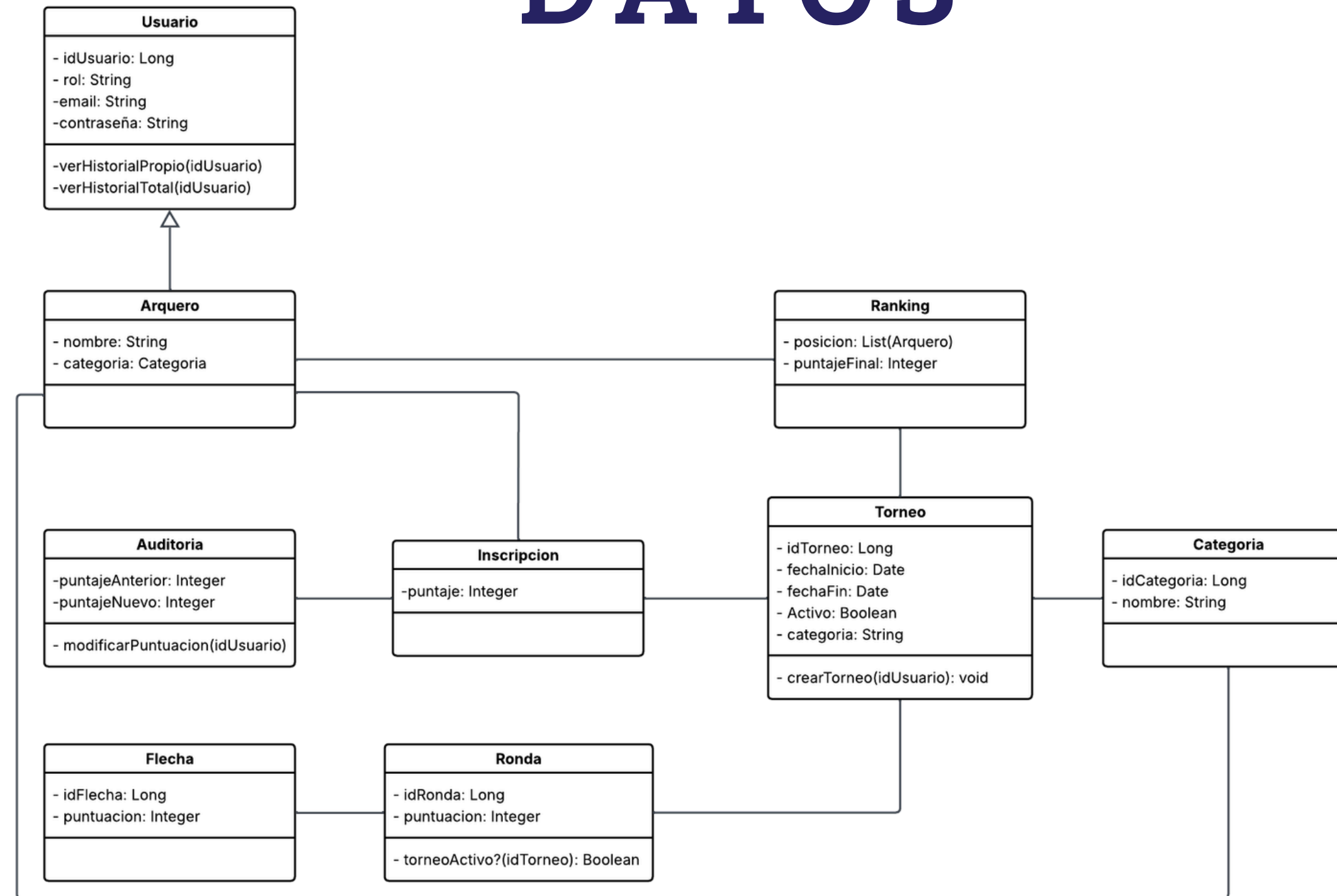
ARQUITECTURA DEL SISTEMA



MODELO DE BASE DE DATOS



MODELO UML DE BASE DE DATOS





LOGICA DE NEGOCIOS DE LA BASE DE DATOS

TRIGGERS

TRIGGERS

TRIGGER 1 - VALIDAR PUNTUACION

**Verifica que la puntuacion del torneo sea valida.
Entero positivo de 0 a 10**

```
-- =====  
-- 3. TRIGGER 1: Validate score [0, 10]  
-- =====  
  
CREATE OR REPLACE FUNCTION fn_validate_arrow_score()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF NEW.score < 0 OR NEW.score > 10 THEN  
        RAISE EXCEPTION 'Puntuación inválida: %. Debe estar entre 0 y 10.', NEW.score;  
    END IF;  
    RETURN NEW;  
END;  
$$;  
  
DROP TRIGGER IF EXISTS trg_validate_arrow_score ON arrows;  
CREATE TRIGGER trg_validate_arrow_score  
BEFORE INSERT OR UPDATE ON arrows  
FOR EACH ROW  
EXECUTE FUNCTION fn_validate_arrow_score();
```

TRIGGERS

TRIGGER 2 - Auditoria de puntuacion

Verifica que la puntuacion del torneo sea valida,

```
140 -- =====
141 -- 4. TRIGGER 2: Score Modification Audit
142 -- =====
143 CREATE OR REPLACE FUNCTION fn_audit_score_modification()
144 RETURNS TRIGGER
145 LANGUAGE plpgsql
146 AS $$
147 DECLARE
148     v_round_record RECORD;
149     v_modified_by BIGINT;
150     v_user_setting TEXT;
151 BEGIN
152     -- Skip audit if score didn't change (UPDATE with same value)
153     IF TG_OP = 'UPDATE' AND OLD.score = NEW.score THEN
154         RETURN NEW;
155     END IF;
156
157     -- Get the User ID from the session variable (set by Spring before the CALL)
158     v_user_setting := current_setting('app.current_user_id', true);
159     IF v_user_setting IS NOT NULL AND v_user_setting <> '' THEN
160         v_modified_by := v_user_setting::BIGINT;
161     ELSE
162         v_modified_by := NULL; -- log without admin ID rather than crashing
163     END IF;
```

```
164
165     -- Get tournament and archer from the round associated with the arrow
166     SELECT id_tournament, id_archer INTO v_round_record
167     FROM rounds
168     WHERE id_round = NEW.id_round;
169
170     -- Insert into audit_log table
171     -- For INSERT: old_score = 0 (arrow is new)
172     -- For UPDATE: old_score = previous value
173     INSERT INTO audit_log (id_archer, id_tournament, old_score, new_score, modified_by, modified_at)
174     VALUES (
175         v_round_record.id_archer,
176         v_round_record.id_tournament,
177         CASE WHEN TG_OP = 'INSERT' THEN 0 ELSE OLD.score END,
178         NEW.score,
179         v_modified_by,
180         CURRENT_TIMESTAMP
181     );
182
183     RETURN NEW;
184 END;
185 $$;
```

STORE PROCEDURES

SP 1 - REGISTRAR PUNTUACION RONDA

Valida torneo activo, crea ronda e inserta todas las fechas en transaccion atomica

```
11  -- =====
12  -- 1. STORED PROCEDURE 1: Register Round Score
13  -- =====
14  CREATE OR REPLACE PROCEDURE registrar_puntuacion_ronda(
15      p_tournament_id BIGINT,
16      p_archer_id BIGINT,
17      p_round_number INT,
18      p_scores INTEGER[]
19  )
20  LANGUAGE plpgsql
21  AS $$
22  DECLARE
23      v_is_active BOOLEAN;
24      v_round_id BIGINT;
25      v_score INTEGER;
26      v_arrow_index INT := 1;
27  BEGIN
28      -- 1. Validate that tournament exists and is active
29      SELECT is_active INTO v_is_active FROM tournaments WHERE id_tournament = p_tournament_id;
30      IF NOT FOUND THEN
31          RAISE EXCEPTION 'Torneo con id % no existe.', p_tournament_id;
32      END IF;
33
34      IF NOT v_is_active THEN
35          RAISE EXCEPTION 'El torneo % no está activo.', p_tournament_id;
36      END IF;
37
38      -- 2. Validate that the archer exists
39      IF NOT EXISTS (SELECT 1 FROM archers WHERE id_archer = p_archer_id) THEN
40          RAISE EXCEPTION 'Arquero con id % no existe.', p_archer_id;
41      END IF;
```

```
42
43      -- 3. Check if the round already exists for the tournament, archer and round number; if not, create
44      SELECT id_round INTO v_round_id
45      FROM rounds
46      WHERE id_tournament = p_tournament_id
47           AND id_archer = p_archer_id
48           AND round_number = p_round_number
49      LIMIT 1;
50
51      IF NOT FOUND THEN
52          INSERT INTO rounds (id_tournament, id_archer, round_number)
53          VALUES (p_tournament_id, p_archer_id, p_round_number)
54          RETURNING id_round INTO v_round_id;
55      END IF;
56
57      -- 4. Validate that no arrows have been registered for this round yet
58      IF EXISTS (SELECT 1 FROM arrows WHERE id_round = v_round_id) THEN
59          RAISE EXCEPTION 'Las flechas de la ronda % para el arquero % ya fueron registradas.', p_round_number, p_archer_id;
60      END IF;
61
62      -- 5. Insert each arrow
63      FOREACH v_score IN ARRAY p_scores
64      LOOP
65          INSERT INTO arrows (id_round, arrow_number, score)
66          VALUES (v_round_id, v_arrow_index, v_score);
67          v_arrow_index := v_arrow_index + 1;
68      END LOOP;
69
70      RAISE NOTICE 'Ronda % registrada para arquero % en torneo %.', p_round_number, p_archer_id, p_tournament_id;
71  END;
72  $$;
73
```

STORE PROCEDURES

SP 2 - CALCULAR RANKING TORNEO

Suma puntajes por arquero y hace upsert en tabla rankings

```
74 -- =====
75 -- 2. STORED PROCEDURE 2: Calculate Tournament Ranking
76 -- =====
77 CREATE OR REPLACE PROCEDURE calcular_ranking_torneo(
78     p_tournament_id BIGINT
79 )
80 LANGUAGE plpgsql
81 AS $$
82 DECLARE
83     v_is_active BOOLEAN;
84     rec RECORD;
85     v_position INT := 1;
86 BEGIN
87     -- 1. Validate Tournament
88     SELECT is_active INTO v_is_active FROM tournaments WHERE id_tournament = p_tournament_id;
89     IF NOT FOUND THEN
90         RAISE EXCEPTION 'Torneo con id % no existe.', p_tournament_id;
91     END IF;
92
93     IF v_is_active THEN
94         RAISE WARNING 'El torneo % sigue activo. Cálculo parcial.', p_tournament_id;
95     END IF;
96
```

```
96
97     -- 2. Clear previous ranking
98     DELETE FROM rankings WHERE id_tournament = p_tournament_id;
99
100    -- 3. Calculate total sum and re-insert
101    FOR rec IN
102        SELECT
103            r.id_archer,
104            SUM(a.score) AS total_score
105        FROM rounds r
106        JOIN arrows a ON a.id_round = r.id_round
107        WHERE r.id_tournament = p_tournament_id
108        GROUP BY r.id_archer
109        ORDER BY total_score DESC
110    LOOP
111        INSERT INTO rankings (id_tournament, id_archer, position, total_score, updated_at)
112        VALUES (p_tournament_id, rec.id_archer, v_position, rec.total_score, CURRENT_TIMESTAMP);
113
114        v_position := v_position + 1;
115    END LOOP;
116 END;
117 $$;
118
```

VISTA MATERIALIZADA

LEADERBOARD HISTORICO

Top 50 arqueros por promedio de puntos/flechas

```
-- =====
-- 5. MATERIALIZED VIEW: Historical Leaderboard
-- =====

CREATE MATERIALIZED VIEW IF NOT EXISTS mv_leaderboard_historico AS
SELECT
    ROW_NUMBER() OVER (ORDER BY ROUND(AVG(a.score)::NUMERIC, 4) DESC) AS posicion_global,
    ar.id_archer,
    ar.name AS nombre,
    COUNT(DISTINCT r.id_tournament) AS torneos_jugados,
    COUNT(a.id_arrow) AS flechas_lanzadas,
    SUM(a.score) AS puntaje_total,
    ROUND(AVG(a.score)::NUMERIC, 4) AS promedio_por_flecha
FROM archers ar
JOIN rounds r ON r.id_archer = ar.id_archer
JOIN arrows a ON a.id_round = r.id_round
GROUP BY ar.id_archer, ar.name
ORDER BY promedio_por_flecha DESC
LIMIT 50
WITH DATA;

CREATE UNIQUE INDEX IF NOT EXISTS uidx_mv_leaderboard_arquero
ON mv_leaderboard_historico (id_archer);
```


VISTA MATERIALIZADA

REFRESCO PROGRAMADO

REFRESH MATERIALIZED VIEW diario o tras cada torneo

```
-- =====  
-- 6. DAILY REFRESH FUNCTION  
-- =====  
  
CREATE OR REPLACE FUNCTION refresh_historical_leaderboard()  
RETURNS VOID  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    REFRESH MATERIALIZED VIEW CONCURRENTLY mv_leaderboard_historico;  
    RAISE NOTICE 'mv_leaderboard_historico refrescada a las %', NOW();  
END;  
$$;
```

API REST Y SEGURIDAD



Método	Ruta	Rol	Descripción
POST	/api/auth/login	Público	Autenticarse y obtener JWT
GET	/api/archers	AUTH	Listar arqueros
POST	/api/archers	ADMIN	Crear arquero
GET	/api/archers/me/history	ARQUER	Historial del arquero autenticado
GET	/api/archers/top-month	Público	Top arqueros del mes
GET	/api/tournaments/{id}/pod	Público	Podio del torneo
POST	/api/rounds	ADMIN	Registrar ronda completa
POST	/api/inscriptions	ADMIN	Inscribir arquero en torneo
GET	/api/audit	ADMIN	Ver log de auditoría

EJECUCION DE LA APLICACION



Contribuciones





MUCHAS
GRACIAS